

Detecting Attacks with Supervised and Unsupervised Learning

Kyle Macadaeg

19 December 2025

1 Abstract

The dataset¹ of interest in this project contains login information for different sessions, some of which have been classified as an attack. As such, the goal is to predict whether there is an attack using the login information; ideally, we could use such a system to identify an attack before it happens.

The features that we have are as follows: session ID, network packet size, protocol type (TCP, UDP, ICMP), login attempts, session duration, encryption (AES, DES, None), IP reputation, failed logins, browser type (Edge, Chrome, Firefox, Unknown), unusual time access (1 or 0).

We can formulate this as a traditional machine learning problem, but this specific setting gives rise to a few unique challenges. We can think of cybersecurity as an arms race between attackers and defenders; as new methods of attack are developed, so too are ways to counter them. We thus have a constantly changing landscape. As a result, we should keep the following in mind when designing our models:

- New data is important to collect for this reason, as it'll give us an up to date look at what an attack looks like. Given that this data is digital, it is also easy to collect. As a consequence, we would like our models to be fast, so that it can be easily retrained on new data. However, a lot of the data in the dataset is categorical, necessitating the use of one-hot vectors, which contributes heavily to the dimensionality of the data. As such, we would like to investigate the relationship between model performance and dimensionality reduction; we hope to find that dimensionality reduction has only a minor effect on performance.
- Since what an attack "looks like" might be different from one time period to the next, it might be helpful to create a model that is able to identify attacks without knowing what a typical attack looks like; only what a typical non-attack looks like. This would require unsupervised learning.

2 Supervised Learning and Dimensionality Reduction

Before anything else, we split the data into a training set and test set. The training set was further divided using K-Folds cross-validation, with 5 folds. Each of the following supervised models were evaluated using these folds.

2.1 No Dimensionality Reduction

First, we need to use supervised learning to make predictions on the dataset without using dimensionality reduction to serve as a baseline. We will test two supervised learning algorithms here. Within the data set as a whole, we have 5273 instances of no attack and 4264 instances of attack. We see that the dataset is relatively balanced, so accuracy will be a good metric on which to evaluate our models. We will also check precision, recall, and F1 score. The two algorithms we are going to test here are:

- Parzen windows using Gaussian kernels (using scikit-learn's KernelDensity).
- Feedforward neural network with one hidden layer (using scikit-learn's MLPClassifier).

One important aspect to note is that we do not want the validation data to affect the training process at all, requiring us to preprocess the training and validation sets of each fold using the means and standard deviations calculated from the training set alone. The following tables summarize our results:

KDE	Accuracy	Precision	Recall	F1
Fold 0	0.8141	0.8619	0.7064	0.7765
Fold 1	0.8092	0.8596	0.6958	0.7684
Fold 2	0.8111	0.8635	0.6890	0.7664
Fold 3	0.8125	0.8317	0.7038	0.7624
Fold 4	0.8042	0.8621	0.6672	0.7522

NN	Accuracy	Precision	Recall	F1
Fold 0	0.8756	0.9375	0.7798	0.8514
Fold 1	0.8735	0.9243	0.7868	0.8500
Fold 2	0.8692	0.9270	0.7698	0.8411
Fold 3	0.8811	0.9282	0.7823	0.8490
Fold 4	0.8685	0.9244	0.7677	0.8388

2.2 LDA

Since LDA can itself be used as a classification algorithm, it is a logical choice of algorithm for dimensionality reduction since it will already try to separate the two classes. By necessity, since we have two classes, we are projecting to a one-dimensional subspace. The following two tables summarize our results:

KDE	Accuracy	Precision	Recall	F1
Fold 0	0.7470	0.7386	0.6911	0.7141
Fold 1	0.7372	0.7254	0.6810	0.7025
Fold 2	0.7448	0.7317	0.6827	0.7064
Fold 3	0.7392	0.7188	0.6399	0.6771
Fold 4	0.7399	0.7234	0.6735	0.6976

NN	Accuracy	Precision	Recall	F1
Fold 0	0.7540	0.7755	0.6498	0.7071
Fold 1	0.7414	0.7554	0.6396	0.6927
Fold 2	0.7434	0.7584	0.6299	0.6882
Fold 3	0.7378	0.7448	0.5876	0.6569
Fold 4	0.7441	0.7542	0.6311	0.6872

We see that the quality of our models has moderately decreased. We still have relatively strong models, but not quite as strong as without dimensionality reduction. There are two things of note about these reduced-dimension models:

- The Parzen window algorithm’s recall has stayed mostly the same between LDA and no LDA. If we think in terms of risk rather than classification error, we can argue that the risk of misclassifying a non-attack is greater than the risk of misclassifying an attack. The former may lead to just an inconvenience, while the latter can lead to a significant security breach. Thus, we may value recall over precision.
- Reducing dimension leads to much reduced runtime. The neural network without LDA took over one minute to process all of the folds, whereas the neural network after LDA only required 8 seconds. Given that the neural network with LDA is still relatively performant, we see that we are getting a large improvement in speed.

2.3 PCA

One flaw with the LDA approach is that it only lets us project to a one-dimensional subspace as we only have two features. Now we use PCA, which allows us to keep a few more dimensions after reduction. Here, we choose to retain 5 components. Once again, we use the same two models as we have before:

KDE	Accuracy	Precision	Recall	F1
Fold 0	0.8588	0.9631	0.7187	0.8231
Fold 1	0.8539	0.9643	0.7055	0.8149
Fold 2	0.8545	0.9677	0.6998	0.8123
Fold 3	0.8727	0.9613	0.7316	0.8301
Fold 4	0.8469	0.9524	0.6907	0.8007

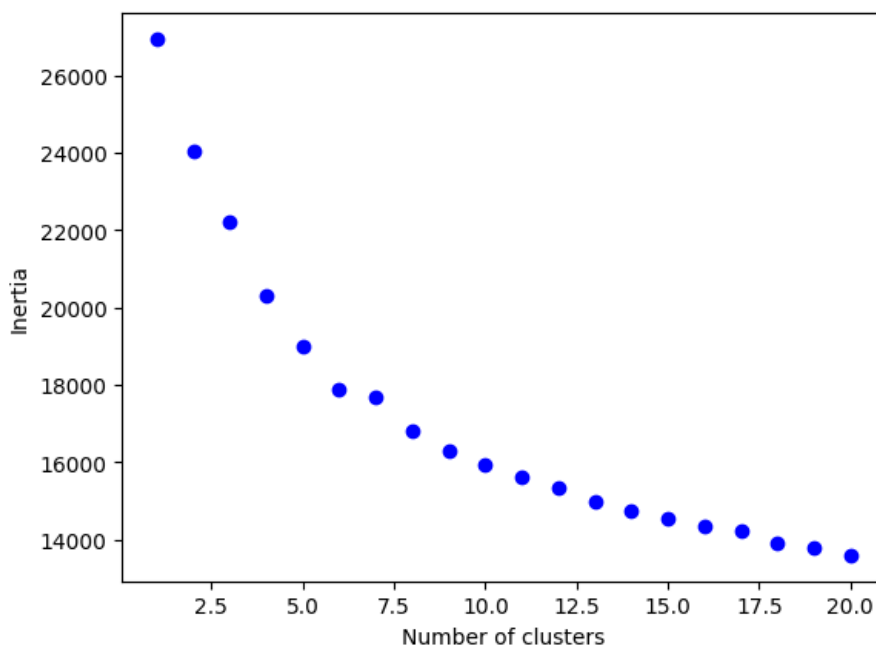
NN	Accuracy	Precision	Recall	F1
Fold 0	0.8665	0.9874	0.7171	0.8308
Fold 1	0.8700	0.9957	0.7178	0.8342
Fold 2	0.8664	0.9892	0.7107	0.8271
Fold 3	0.8783	0.9802	0.7300	0.8368
Fold 4	0.8622	0.9825	0.7033	0.8198

Surprisingly, the Parzen windows approach appears to be doing better after PCA. The neural network performs slightly worse after PCA, but it is able to process all folds in about half the time.

3 Unsupervised Learning

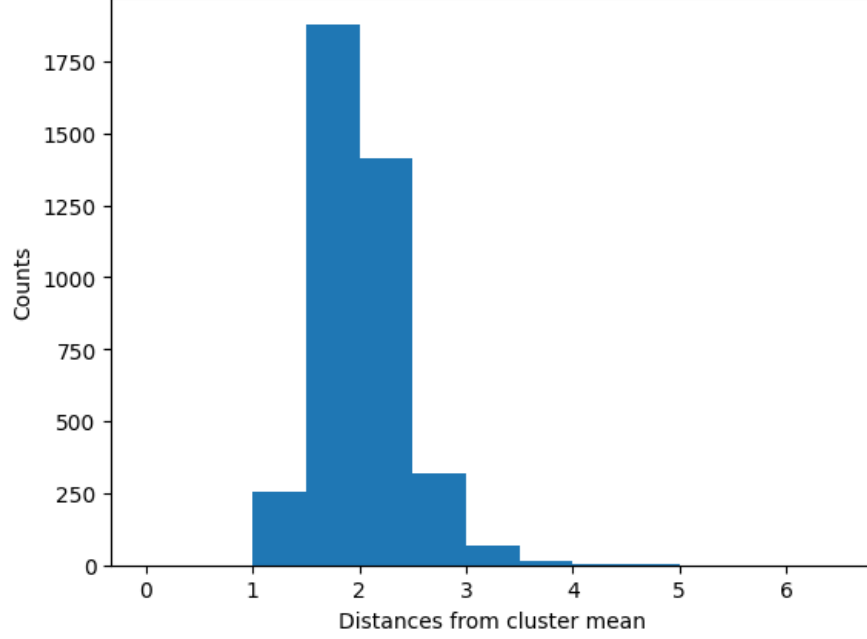
Our next goal is to address the second point of the abstract. Since methods of attack might change with time, we would like to create models that flag suspicious activity rather than learn to differentiate attacks from non-attacks using existing data. In other words, we would like to train a model on known non-attacks only. This calls for unsupervised learning; here, we use the K-Means clustering algorithm.

After we remove attacks from our training data, we need to determine a suitable number of clusters. To do this, we use K-Means several times with different settings of the hyperparameter, and graph the number of clusters versus inertia², the sum of squared distances from each point to the center of their cluster. This produces the following graph:



After $k = 8$, we start to see diminishing returns, so that is our choice of hyperparameter.

Once we have our centers, our next goal is to get an idea of the distribution of distances of points from their respective means. Here, we have a histogram of these distances.



We will define our algorithm as follows. For a percentile threshold p , we will calculate the p 'th percentile in distance d . For example, for $p = 90$, we get the number $d = 2.5111$, signifying that 90% of points are at most 2.5111 away from their respective cluster means. For a point we would like to predict, we will check its distance to the nearest cluster mean. If it is greater than d , we guess that it is an attack. Our results are summarized in the following table:

	Accuracy	Precision	Recall	F1
$p = 60, d = 2.0577$	0.6847	0.6080	0.8313	0.7023
$p = 70, d = 2.1699$	0.7338	0.6693	0.8004	0.7290
$p = 80, d = 2.3008$	0.7686	0.7404	0.7432	0.7418
$p = 90, d = 2.5111$	0.7790	0.8261	0.6410	0.7219

As expected, our model struggles more to classify attacks than the supervised model, but it still performs relatively well.

4 Conclusion and Results

At this point, we train each of our supervised models on the entire training set and evaluate them on the test set. We see that the performance of each model is similar to what we have seen during validation (please see the conclusion section of the ipynb file for details). From this, we can conclude the following.

- A high degree of efficiency can be obtained by sacrificing only a small amount of performance. The neural network model performs a bit worse on reduced-dimension data, but at a much faster speed. We also saw that dimensionality reduction can actually *improve* performance, as was seen for PCA and the Parzen window method.
- Unsupervised learning can be used to get a successful model while only relying on data with label 0, showing that a model does not need to see what a typical attack looks like to identify an attack.

5 References

1. <https://www.kaggle.com/datasets/dnkumars/cybersecurity-intrusion-detection-dataset>
2. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KernelDensity.html>